Shafran Yuriy

C++ Systems and Compiler Programming Student at MIPT (2nd year)

+7 922 093-87-19

y.shafran-06@mail.ru

Dolgoprudny, Russia

@atlaxsyys

github.com/Atlaxsyys

Education

Moscow Institute of Physics and Technology (MIPT) **2024–Present**
Department of Radio Engineering and Computer Science
GPA: 7.86

Systems Programming and Compiler Technologies (ILab by I.R. Dedinskiy) 1st–2nd semester

Algorithms and Data Structures (VK Education) 3rd semester **[certificate]**

MIPT ILab C++ Course (Vladimirov K.I.) 3rd semester – Present

Business Programming Paradigms: Databases and SQL (MIPT) 3rd semester – Present

Introduction to Tensor Compilers (MIPT) In progress

Introduction to the Architecture of Computing Systems (MIPT) In progress

Projects

ParaCL (C++) [GitHub]
Team project: Developed a compiler and interpreter for a C-like language using Flex and Bison. Implemented AST-based execution via tree traversal and built an LLVM IR backend for code generation.

Tensor Compiler (C++) [GitHub]
Team project: Developing a tensor compiler with ONNX frontend and custom graph-based IR. Implemented model import, graph construction and validation, and visualization pipeline using Graphviz. Backend is currently under development.

AngryMIPTs (C++) [GitHub]
Team project: Developed a physics-based projectile game (similar to Angry Birds) using Box2D and SFML. Implemented a multithreaded architecture with lock-free command/event queues and double-buffered world state. Built backend services in Go with PostgreSQL for authentication and leaderboards.

Bitonic Sort on OpenCL (C++) [GitHub]
Team project: Implemented a GPU-accelerated Bitonic Sort using OpenCL, including host-device pipeline, kernel execution, and memory management. Built benchmarking and testing framework and analyzed performance against `std::sort`.

2Q Cache (C++) [GitHub]
Implemented a 2Q caching algorithm with three-queue architecture (A1in, A1out, Am) to improve cache hit rates. Built testing framework and compared performance against an ideal cache model.

Akinator (C) [GitHub]
Implemented a decision-tree-based Akinator game in C. Designed dynamic tree updates with user-defined nodes, file persistence, and Graphviz-based visualization of the knowledge base.

Technical Skills

Programming: C, C++, Python (basic), Bash
Technologies: LLVM IR, OpenCL, Flex, Bison, PostgreSQL, Box2D, SFML, Graphviz
Tools: Git, Make, CMake, GTest, Linux, GitHub Actions
Languages: Russian (native), English (B1–B2)